

g·NEEDACCESS
NETWORK ENABLED EASY DATA ACCESS

USER MANUAL

MATLAB API





User Manual: g.NEEDaccess MATLAB API
Version: 1.24.00

Copyright © 2024 g.tec medical engineering GmbH
www.gtec.at | office@gtec.at

CONTENTS

1	G.TEC DEVICE INTERFACE CLASS PROPERTIES AND METHODS	5
1.1	COMMON GDS PROPERTIES	5
1.2	DEVICE SPECIFIC CLASSES AND THEIR PROPERTIES.....	5
1.2.1	G.HIAMP (GHIAMPDEVICECONFIGURATION)	5
1.2.2	G.NAUTILUS (GNAUTILUSDEVICECONFIGURATION)	7
1.2.3	G.USBAMP (GUSBAMPDEVICECONFIGURATION)	8
1.3	SUPPORTED SAMPLING RATES	9
1.4	GET DEVICES CURRENTLY CONNECTED TO THE COMPUTER	10
1.4.1	GETCONNECTEDDEVICES	10
1.5	MEASURE IMPEDANCE	11
1.5.1	GETIMPEDANCE.....	11
1.6	COMMON METHODS TO GET DEVICE CAPABILITIES	12
1.6.1	GETAVAILABLECHANNELS	12
1.7	GETAVAILABLEFILTERS.....	13
1.7.1	GETSUPPORTEDSAMPLINGRATES	14
1.7.2	GETDEVICEINFO.....	14
1.7.3	GETSCALING	15
2	DEVICE SPECIFIC METHODS	16
2.1	G.NAUTILUS METHODS	16
2.1.1	GETSUPPORTEDSENSITIVITIES.....	16
2.1.2	GETSUPPORTEDINPUTSOURCES.....	16
2.1.3	GETCHANNELNAMES	16
2.1.4	GETSUPPORTEDNETWORKCHANNELS	17
2.1.5	GETNETWORKCHANNEL.....	17
2.1.6	SETNETWORKCHANNEL	17
2.1.7	GETNUMBEROFMOUNTEDMODULES	18
2.1.8	GETAVAILABLEDIOS	18
2.2	G.USBAMP METHODS	18
2.2.1	GETASYNCDIGITALIOS	18
2.2.2	SETASYNCDIGITALOUTPUTS	19
3	METHODS FOR DATA ACQUISITION	20
3.1	SETCONFIGURATION.....	20
3.2	STARTDATAACQUISITION	20
3.3	STARTSTREAMING	20
3.4	STOPSTREAMING	20
3.5	STOPDATAACQUISITION.....	20
3.6	GETDATA	20
4	QUICK START	22

4.1	G.NAUTILUS	22
4.2	G.HIAMP	23
4.3	G.USBAMP	25
5	DIGITAL INPUTS	27
5.1	G.NAUTILUS	27
5.2	G.HIAMP	29
5.3	G.USBAMP	30
6	DISPLAY DATA	32
6.1	G.NAUTILUS	32
6.2	G.HIAMP	34
6.3	G.USBAMP	37
7	SYNCHRONIZING G.USBAMP	40
8	HELP	42

1 g.tec DEVICE INTERFACE CLASS PROPERTIES AND METHODS

The g.tec device interface MATLAB class `gtecDeviceInterface` provides an interface to all applicable settings for g.NEEDaccess and the g.tec devices g.HIamp, g.Nautilus or g.USBamp in their corresponding device classes, as well as all methods provided by GDS to get the corresponding amplifier capabilities.

1.1 COMMON GDS PROPERTIES

<code>IPAddressHost</code>	Define IP address of server where the g.tec device is connected as string
<code>HostPort</code>	Port of host connection as integer
<code>IPAddressLocal</code>	Define IP address of local computer as string
<code>LocalPort</code>	Port of local connection as integer
<code>DeviceConfigurations</code>	Holding the device specific configurations for g.HIamp, g.Nautilus or g.USBamp.

Properties available during data acquisition

<code>SamplesAvailable</code>	Holds the number of samples currently available. Read only, valid only when data acquisition is running.
<code>isRunning</code>	Information that data acquisition is running. Read only, true when data is acquired, false otherwise.

1.2 DEVICE SPECIFIC CLASSES AND THEIR PROPERTIES

1.2.1 G.HIAMP (GHIAMPDEVICECONFIGURATION)

Amplifier Settings

<code>Name</code>	String holding the serial number of g.HIamp (e.g. HA-2014.01.02)
<code>DeviceType</code>	String representing the device type (GHIamp) (predefined, must not be changed)
<code>SamplingRate</code>	Specify the sampling frequency of g.HIamp in Hz as unsigned integer
<code>NumberOfScans</code>	Specify the buffering block size as unsigned short, possible values depend on sampling rate, use function <code>GetSupportedSamplingRates</code> to get recommended values.
<code>Channels</code>	Array of g.HIamp channel configurations (<code>gHIampChannels</code>) holding properties for each analog channel

Options

CounterEnabled	Show a counter on first recorded channel which is incremented with every block transmitted to the PC. Overruns at 1000000.
TriggerLinesEnabled	Scan the digital trigger channel with the analog inputs
HoldEnabled	enable signal hold
InternalSignalGenerator	Holds a gHlampInternalSignalGenerator object to enable and configure internal signal generator of g.Hlamp

Internal Signal Generator (gHlampInternalSignalGenerator)

Enabled	Apply internal test signal to all inputs (requires shortcut of all analog channels to ground)
Amplitude	Amplitude of the test signal (7.62283 mV fix)
Offset	Offset of the test signal (-7.62283 mV fix)
Frequency	Specify the frequency of the test signal

Channel Settings (gHlampChannels)

ChannelNumber	Unsigned integer holding the channel number of the analog channel
Available	Bool value indicating availability of an analog channel (read only)
Acquire	Bool value selecting the channel for data acquisition
BandpassFilterIndex	Perform a digital bandpass filtering of the input channels. Use GetAvailableFilters to get filter indices.
NotchFilterIndex	Perform a bandstop filtering to suppress the power line frequency of 50 Hz or 60 Hz. Use GetAvailableFilters to get filter indices.
ReferenceChannel	Select a channel number as reference channel for an analog channel

1.2.2 G.NAUTILUS (GNAUTILUSDEVICECONFIGURATION)

Amplifier Settings	
Name	String holding the serial number of g.Nautilus (e.g. NA-2014.07.67)
DeviceType	String representing the device type (GNautilus)
SamplingRate	Specify the sampling frequency of g.Nautilus in Hz as unsigned integer
NumberOfScans	Specify the buffering block size as unsigned short, possible values depend on sampling rate, use function GetSupportedSamplingRates to get recommended values.
InputSignal	holding type of input signal, can be 1 (Electrode), 2 (Shortcut) or 6 (TestSignal)
Channels	Array of g.Nautilus channel configurations (gNautilusChannels) holding properties for each analog channel

Options	
NoiseReduction	Bool value enabling noise reduction for g.Nautilus
CAR	Bool value enabling common average calculation for g.Nautilus
AccelerationData	Bool value enabling acquisition of acceleration data from g.Nautilus head stage, adds 3 additional channels to the data acquisition for x, y, and z direction
Counter	show a counter as an additional channel
LinkQualityInformation	Bool value enabling additional channel informing about link quality between head stage and base station
BatteryLevel	Bool to enable acquisition of additional channel holding information about remaining battery capacity
DigitalIOs	scan the digital channels with the analog inputs and add it as additional channel acquired
ValidationIndicator	Enables the additional channel validation indicator, informing about the liability of the data recorded
NetworkChannel	Unsigned integer value representing the network channel used between head stage and base station

Specify Channel Settings (gNautilusChannels)

ChannelNumber	Unsigned integer holding the channel number of the analog channel
Available	Bool value indicating availability of an analog channel
Acquire	Bool value selecting the channel for data acquisition
Sensitivity	Double value representing the sensitivity of the specified channel
UsedForNoiseReduction	Bool value indicating if channel should be used for noise reduction
UsedForCAR	Bool value indicating if channel should be used for common average calculation
BandpassFilterIndex	Perform a digital bandpass filtering of the input channels. Use GetAvailableFilters to get filter indices.
NotchFilterIndex	Perform a bandstop filtering to suppress the power line frequency of 50 Hz or 60 Hz. Use GetAvailableFilters to get filter indices.
BipolarChannel	Select a zero based channel index as reference channel for an analog channel

1.2.3 G.USBAMP (GUSBAMPDEVICECONFIGURATION)

Amplifier Settings

Name	String holding the serial number of g.USBamp (e.g. UB-2014.05.23)
DeviceType	String representing the device type (GUsbAmp)
SamplingRate	Specify the sampling frequency of g.Hlamp in Hz as unsigned integer
NumberOfScans	Specify the buffering block size as unsigned short, possible values depend on sampling rate, use function GetSupportedSamplingRates to get recommended values.
CommonGround	Array of 4 bool elements to enable or disable common ground
CommonReference	Array of 4 bool values to enable or disable common reference
Channels	Array of g.USBamp channel configurations (gUSBampChannels) holding properties for each analog channel

Options	
ShortCutEnabled	Bool enabling or disabling g.USBamp shortcut
CounterEnabled	Show a counter on first recorded channel which is incremented with every block transmitted to the PC. Overruns at 1000000.
TriggerEnabled	scan the digital trigger channel with the analog inputs
InternalSignalGenerator	Holds a gUSBampInternalSignalGenerator object to enable and configure internal signal generator of g.USBamp

Internal Signal Generator (gUSBampInternalSignalGenerator)	
Enabled	Apply internal test signal to all inputs
WaveShape	Unsigned integer representing the wave shape of the internal test signal. Can be 1 (square), 2 (saw tooth), 3 (sine) 4 (DRL) or 5 (noise)
Amplitude	the amplitude of the test signal (can be -250 to 250 mV)
Offset	the offset of the test signal (can be -200 to 200 mV)
Frequency	specify the frequency of the test signal (can be 1 to 100 Hz)

Specify Channel Settings (gUSBampChannels)	
ChannelNumber	Unsigned integer holding the channel number of the analog channel
Available	Bool value indicating availability of an analog channel (read only)
Acquire	Bool value selecting the channel for data acquisition
BandpassFilterIndex	Perform a digital bandpass filtering of the input channels. Use GetAvailableFilters to get filter indices.
NotchFilterIndex	Perform a bandstop filtering to suppress the power line frequency of 50 Hz or 60 Hz. Use GetAvailableFilters to get filter indices.
BipolarChannel	Select a channel number as reference channel for an analog channel

1.3 SUPPORTED SAMPLING RATES

For the relation between number of scans and sampling rate it is generally recommended to have a ratio of minimum 30 ms (see formula below)

$$30 \text{ ms} \leq \frac{\text{Number of Scans}}{\text{Sampling Rate}} * 1000$$

For the standard recommended settings see table below.

g.HIamp		g.USBamp		g.Nautilus	
Sampling Rate [Hz]	Recommended Number of Scans	Sampling Rate [Hz]	Recommended Number of Scans	Sampling Rate [Hz]	Recommended Number of Scans
256	8	32	1	250	8
512	16	64	2	500	15
600	32	128	4		
1200	64	256	8		
2400	128	512	16		
48000	256	600	32		
9600	256	1200	64		
19200	256	2400	128		
38400	256	4800	256		
		9600	512		
		19200	512		
		38400	512		

Settings below 30 ms are likely to lose data during data acquisition.

1.4 GET DEVICES CURRENTLY CONNECTED TO THE COMPUTER

This function returns all g.tec devices available at a defined endpoint (IP address and port). Host and local IP and corresponding ports are required to use this function (see common GDS properties section).

1.4.1 GETCONNECTEDDEVICES

Gets a structure with all devices currently connected to the computer at the defined IP address.

Call with:

```
connected_devices = gds_interface.GetConnectedDevices();
```

Type	Name	Description
Input	No input	
Outputs	connected_devices	Structure with one element for each g.tec amplifier found containing three fields each
Output fields	DeviceType	String with type of connected device (GNautilus, GHiAmp or GUSBamp)
	Name	String with serial number of connected device (e.g. NA-2014.07.67)
	InUse	Bool value indicating if device connected is currently used for a data acquisition

1.5 MEASURE IMPEDANCE

g.NEEDaccess allows the measurement of channel impedances for g.Nautilus, g.HIamp and g.USBamp.

1.5.1 GETIMPEDANCE

Returns the impedances for a selected range of channels or for a single channel (g.USBamp) or for all channels available (g.Nautilus and g.HIamp). If more than one g.USBamp is connected, the serial number of the device to measure impedances has to be provided.

g.HIamp distinguishes between active and passive electrodes, it is not possible to measure both though (will throw an exception in this case). If g.HIamp is used with impedance measurement, please use an optional string input either with the string active or passive. See calling example below.

For g.USBamp (single or synchronized) call with:

```
impedances = gds_interface.GetImpedance(gusbamp_config.Name,
channels_selected);
```

For g.HIamp call for passive electrodes with:

```
impedances = gds_interface.GetImpedance('passive');
```

Or for active electrodes with:

```
impedances = gds_interface.GetImpedance('active');
```

For g.Nautilus and also single g.USBamp call with:

```
impedances = gds_interface.GetImpedance();
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp, must be provided if more than one g.USBamp is used.
	<code>channels_selected</code>	g.USBamp only! Array with 1 x m elements containing the channels whose impedance should be measured. Optional, impedances for all channels of the connected device are returned if unused.
		g.HIamp only! String input with 'active' or 'passive', depending on the electrode type used.
Output	<code>impedances</code>	Array with 2 x m elements containing the channel number and the corresponding impedance value



ATTENTION

Function `GetImpedance` might throw error message when no impedance can be measured for the device currently connected, mainly when ground and/or reference electrodes are not connected.

1.6 COMMON METHODS TO GET DEVICE CAPABILITIES

All methods require a device configuration holding at least the device serial (Name property in `gNautilusDeviceConfiguration`, `gHIampDeviceConfiguration` or `gUSBampDeviceConfiguration`)



NOTE

If g.USBamp is used and multiple amplifiers are synchronized, each of the functions in this section requires the serial number of the g.USBamp to return the property for this device. Each function will fail otherwise.

1.6.1 GETAVAILABLECHANNELS

Returns a bool array with as many elements as the maximum possible channel number of a g.tec device (64 channels for g.Nautilus, 256 channels for g.HIamp or 16 channels for g.USBamp), where each element having the value true (1) represents an available analog channel of the device currently connected. False (0) represents an unavailable channel.

For g.USBamp call with:

```
available_channels = gds_interface.GetAvailableChannels(gusbamp_config.Name);
```

For g.Nautilus and g.HIamp call with:

```
available_channels = gds_interface.GetAvailableChannels();
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp, only valid if g.USBamp is used.
Output	<code>available_channels</code>	Array of bool with 16 (g.USBamp), 64 (g.Nautilus) or 256 (g.Hlamp) elements, where true represents an hardware available channel, false otherwise.

1.7 GETAVAILABLEFILTERS

Gets a structure containing all bandpass and notch filters available for the current device.

For g.USBamp call with:

```
available_filterss =  
gds_interface.GetAvailableFilters(gusbamp_config.Name, sampling_rate);
```

For g.Nautilus and g.Hlamp call with:

```
available_filters = gds_interface.GetAvailableFilters(sampling_rate);
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp, only valid if g.USBamp is used.
	<code>sampling_rate</code>	Sampling rate to return filters for, optional
Output fields	<code>available_filters</code>	Structure with two sub-structures, one for available bandpass filters and one for available notch filters.
	<code>BandpassFilters</code>	Structure with one element for each bandpass filter available containing five fields each
	<code>NotchFilters</code>	Structure with one element for each notch filter available containing five fields each
	<code>FilterIndex</code>	Integer value holding the index for the device setting in the analog channel configuration of an amplifier either <code>BandpassFilterIndex</code> or <code>NotchFilterIndex</code> (please see <code>gNautilusChannels</code> , <code>gHIampChannels</code> or <code>gUSBampChannels</code> definition for further information)
	<code>SamplingRate</code>	Double value holding the sampling rate for which the filter is valid
	<code>Order</code>	Unsigned integer holding filter order
	<code>LowerCutoffFrequency</code>	Double representing lower cutoff frequency of the filter
	<code>UpperCutoffFrequency</code>	Double representing upper cutoff frequency of the filter
	<code>TypeId</code>	String representing type of filter

1.7.1 GETSUPPORTEDSAMPLINGRATES

Returns an array of unsigned integers holding the sampling rates supported by the connected device.

For g.USBamp call with:

```
supported_fs = gds_interface.GetSupportedSamplingRates(gusbamp_config.Name);
```

For g.Nautilus and g.HIamp call with:

```
supported_fs = gds_interface.GetSupportedSamplingRates();
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp, only valid if g.USBamp is used.
Output	<code>supported_fs</code>	Array of unsigned integers with the dimensions 2 x m, where one represents the available sampling rates and the other one the corresponding recommended number of scans.

1.7.2 GETDEVICEINFO

Return a string with the complete device information of the device currently connected.

For g.USBamp call with:

```
device_info = gds_interface.GetDeviceInfo(gusbamp_config.Name);
```

For g.Nautilus and g.Hlamp call with:

```
device_info = gds_interface.GetDeviceInfo();
```

Type	Name	Description
Input	gusbamp_config.Name	Serial number of g.USBamp, only valid if g.USBamp is used.
Output	device_info	String holding the complete hardware information of the connected device.

1.7.3 GETSCALING

This function gets a structure with arrays for scaling factor and offset for each analog channel of the connected device.

For g.USBamp call with:

```
device_scaling = gds_interface.GetScaling(gusbamp_config.Name);
```

For g.Nautilus and g.Hlamp call with:

```
device_scaling = gds_interface.GetScaling();
```

Type	Name	Description
Input	gusbamp_config.Name	Serial number of g.USBamp, only valid if g.USBamp is used.
Output	device_scaling	Structure with two elements, scaling factor and offset
Output fields	Factor	Array holding single type values with scaling factor for each analog channel.
	Offset	Array holding single type values with offset for each analog channel.

2 DEVICE SPECIFIC METHODS

2.1 g.NAUTILUS METHODS

2.1.1 GETSUPPORTEDSENSITIVITIES

Retrieves the supported sensitivity values for a g.Nautilus device.

Call with:

```
supported_sensitivities = gds_interface.GetSupportedSensitivities();
```

Type	Name	Description
Input	No Input	Serial number of g.USBamp, only valid if g.USBamp is used.
Output	supported_sensitivities	Array of double values each one holding a supported sensitivity value for g.Nautilus currently connected.

2.1.2 GETSUPPORTEDINPUTSOURCES

Retrieves the available input sources a g.Nautilus device can use to acquire signals.

Call with:

```
supported_input_sources = gds_interface.GetSupportedInputSources();
```

Type	Name	Description
Input	No Input	
Output	supported_input_sources	Structure holding name and value for each input source available.
Output fields	Name	String describing the input source (e.g. Electrode for the EEG input of g.Nautilus)
	Value	Unsigned integer holding the number of the corresponding input source (e.g. 1 for Electrode)

2.1.3 GETCHANNELNAMES

Gets the names of all available channels depending on the mounted grid.

Call with:

```
channel_names = gds_interface.GetChannelNames();
```


Type	Name	Description
Input	No Input	
Output	channel_names	Cell array holding an element with a channel name for each analog channel available.

2.1.4 GETSUPPORTEDNETWORKCHANNELS

Retrieves the available network channels a g.Nautilus device can use to pair its headset and base station.

Call with:

```
supported_network_channels = gds_interface.GetSupportedNetworkChannels();
```

Type	Name	Description
Input	No Input	
Output	supported_network_channels	Array of 32-bit unsigned integer containing the supported network channels for g.Nautilus.

2.1.5 GETNETWORKCHANNEL

Retrieves the radio channel currently used for wireless communication between headset and base station.

Call with:

```
network_channel = gds_interface.GetNetworkChannel();
```

Type	Name	Description
Input	No Input	
Output	Network_channel	Network channel currently used for communication between g.Nautilus base station and headset.

2.1.6 SETNETWORKCHANNEL

Sets the radio channel used for wireless communication between the headset and base station.

Call with:

```
gds_interface.SetNetworkChannel(network_channel);
```

Type	Name	Description
Input	<code>network_channel</code>	Network channel to be used for communication. Use <code>GetSupportedNetworkChannels</code> to retrieve possible network channels.
Output	No output	

2.1.7 GETNUMBEROFMOUNTEDMODULES

Gets the number of the actually mounted modules of the currently connected g.Nautilus device.

Call with:

```
mounted_modules = gds_interface.GetNumberOfMountedModules();
```

Type	Name	Description
Input	No Input	
Output	<code>mounted_modules</code>	Number of mounted modules on g.Nautilus headstage (corresponds to the number of channels the device offers).

2.1.8 GETAVAILABLEDIOS

Gets information about digital I/O channels available of the specified g.Nautilus device.

Call with:

```
available_dios = gds_interface.GetAvailableDIOs();
```

Type	Name	Description
Input	No Input	
Output	<code>available_dios</code>	Structure containing one element for each digital IO available.
Output fields	<code>ChannelNumber</code>	Unsigned integer representing the digital IO number.
	<code>Direction</code>	String representing the direction of the digital channel (In or Out)

2.2 G.USBAMP METHODS

2.2.1 GETASYNCDIGITALIOS

Use this function to get the asynchronous digital in- or outputs available for g.USBamp currently used for the data acquisition session. This function requires one input argument and has one output argument.

Call with:

```
async_dios = gds_interface.GetAsyncDigitalIOs(gusbamp_config.Name);
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp
Output	<code>async_dios</code>	Structure with as many elements as asynchronous digital IOs are available, each elements contains three fields
Output fields	<code>ChannelNumber</code>	Integer value representing the digital channel number
	<code>Direction</code>	String holding the digital channel direction (In or Out)
	<code>Value</code>	Current value of the digital channel (true or false).

2.2.2 SETASYNCDIGITALOUTPUTS

Use this function to set the asynchronous digital outputs available for the g.USBamp currently used for the data acquisition session. This function requires two input arguments and has no output argument.

Call with:

```
gds_interface.SetAsyncDigitalIOs(gusbamp_config.Name, async_dios);
```

Type	Name	Description
Input	<code>gusbamp_config.Name</code>	Serial number of g.USBamp
	<code>async_dios</code>	Structure with as many elements as asynchronous digital IOs are available, each elements contains three fields
Input fields	<code>ChannelNumber</code>	Integer value representing the digital channel number
	<code>Direction</code>	String holding the digital channel direction (In or Out)
	<code>Value</code>	Value to be applied for the digital channel (true or false).
Output		

Value: Value to be applied for the digital channel to set (true or false).

3 METHODS FOR DATA ACQUISITION

3.1 SETCONFIGURATION

Sets the configuration currently available in the property DeviceConfigurations of `gtecDeviceInterface`.

Input: none.

Output: none.

3.2 STARTDATAACQUISITION

Starts the data acquisition with the settings provided. This start the device data acquisition not streaming to the connected endpoint, which is done by `StartStreaming`.

Input: none.

Output: none.

3.3 STARTSTREAMING

Starts the data stream to the endpoint specified before starting data acquisition.

Input: none.

Output: none.

3.4 STOPSTREAMING

Stops the data stream to the endpoint specified before starting data acquisition.

Input: none.

Output: none.

3.5 STOPDATAACQUISITION

Stops the data acquisition currently running and closes the connection.

Input: none.

Output: none.

3.6 GETDATA

Retrieves data during the data acquisition is running.

Call with:

```
[scans_received, data] = gds_interface.GetData(number_of_scans);
```

Type	Name	Description
Input	number_of_scans	Can either be empty to retrieve the number of scans currently available (0 as input does the same) or if an input is provided should be defined as equal to or a multiple of the number of scans in the device configuration (NumberOfScans property in gNautilusDeviceInterface, gHlampDeviceInterface or gUSBampDeviceInterface).
	async_dios	Structure with as many elements as asynchronous digital IOs are available, each elements contains three fields
Output	scans_received	Actual amount of acquired scans
	data	Acquired data in the form scans x channels, data type returned is single



NOTE

All biosignal data returned by the GetData function is in μV .



ATTENTION

For g.Hlamp and g.USBamp it is strongly recommended to use 0 or an empty input for the GetData function, to return all scans currently available (especially for sampling rates above 2400 Hz). The first value returned by GetData will hold the number of actually returned scans.



ATTENTION

It is strongly advised to use the values for number of scans recommended by GetSupportedSamplingRates for the device configuration and the same value or multiples of it for GetData function to avoid data loss.

4 QUICK START

4.1 g.NAUTILUS



NOTE

The example contained in the g.Nautilus examples installation folder (default C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNautilus) is using a 32 channel g.Nautilus device. If another device is used (8, 16 or 64 channels), the code has to be adapted correspondingly (see comments in the example code).

To test g.NEEDaccess MATLAB API with g.Nautilus on your system, please perform the following example:

1. **Create a gtecDeviceInterface object with**
`gds_interface = gtecDeviceInterface;`

2. **Set the IP and ports for the host and the client. If used on a local computer, set the following parameters**

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. **Get the device(s) currently connected**
`connected_devices = gds_interface.GetConnectedDevices();`

4. **Use gNautilusDeviceConfiguration to create a g.Nautilus device configuration and, as there should only be one g.Nautilus device available, set the device serial as name**

```
gnautilus_config = gNautilusDeviceConfiguration();  
gnautilus_config.Name = connected_devices(1,1).Name;
```

5. **To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object**

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

6. **Get available channels, supported sensitivities and supported input sources**

```
available_channels = gds_interface.GetAvailableChannels();  
supported_sensitivities = gds_interface.GetSupportedSensitivities();  
supported_input_sources = gds_interface.GetSupportedInputSources();
```

7. **Set the sampling rate to 250 Hz and number of scans to 8**

```
gnautilus_config.SamplingRate = 250;  
gnautilus_config.NumberOfScans = 8;
```

8. **Set input source to record the internal test signal of g.Nautilus, a square wave applied to all analog channels with a frequency of 10 Hz and an amplitude of 15 mV**

```
gnautilus_config.InputSource = supported_input_source(3).Value;
```

9. Enable additional channels counter and validation indicator

```
gnautilus_config.Counter = true;  
gnautilus_config.ValidationIndicator = true;
```

10. Record all channels available

```
for i=1:size(gNautilusConfig.Channels,2)  
    if (available_channels(1,i))  
        gnautilus_config.Channels(1,i).Available = true;  
        gnautilus_config.Channels(1,i).Acquire = true;  
        % set sensitivity to 187.5 mV  
        gnautilus_config.Channels(1,i).Sensitivity = ...  
            supported_sensitivities(6);  
        % do not use channel for CAR and noise reduction  
        gnautilus_config.Channels(1,i).UsedForNoiseReduction = false;  
        gnautilus_config.Channels(1,i).UsedForCAR = false;  
        % do not use filters  
        gnautilus_config.Channels(1,i).BandpassFilterIndex = -1;  
        gnautilus_config.Channels(1,i).NotchFilterIndex = -1;  
        % do not use a bipolar channel  
        gnautilus_config.Channels(1,i).BipolarChannel = -1;  
    end  
end
```

11. Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

12. Configure g.Nautilus with the current settings

```
gds_interface.SetConfiguration();
```

13. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();  
gds_interface.StartStreaming();
```

14. Get data acquired from g.Nautilus, 2500 samples for 10 seconds of data

```
samples_acquired = 0;  
while (samples_acquired < 2500)  
    [scans_received, data] = gds_interface.GetData(8);  
    samples_acquired = samples_acquired + scans_received;  
end
```

15. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```
gds_interface.StopStreaming();  
gds_interface.StopDataAcquisition();  
delete(gds_interface);  
clear gds_interface;
```

This example is also available as MATLAB script gNautilusAPIDemo1.m in the folder

C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNautilus

4.2 g.HIAMP

To test g.NEEDaccess MATLAB API with g.HIamp, please perform the following example:

1. Create a gtecDeviceInterface object with

```
gds_interface = gtecDeviceInterface;
```

2. **Set the IP and ports for the host and the client. If used on a local computer, set the following parameters**

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. **Get the device(s) currently connected**

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. **Use gHIampDeviceConfiguration to create a g.HIamp device configuration and, as there should only be one g.HIamp device available, set the device serial as name**

```
ghiamp_config = gHIampDeviceConfiguration();  
ghiamp_config.Name = connected_devices(1,1).Name;
```

5. **To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object**

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

6. **Get available channels**

```
available_channels = gds_interface.GetAvailableChannels();
```

7. **Set the sampling rate to 256 Hz and number of scans to 4**

```
ghiamp_config.SamplingRate = 256;  
ghiamp_config.NumberOfScans = 4;
```

8. **Enable counter**

```
ghiamp_config.CounterEnabled = true;
```

9. **Record all channels available**

```
for i=1:size(ghiamp_config.Channels,2)  
    if (available_channels(1,i))  
        ghiamp_config.Channels(1,i).Available = true;  
        ghiamp_config.Channels(1,i).Acquire = true;  
        % do not use filters  
        ghiamp_config.Channels(1,i).BandpassFilterIndex = -1;  
        ghiamp_config.Channels(1,i).NotchFilterIndex = -1;  
        % do not use a bipolar channel  
        ghiamp_config.Channels(1,i).ReferenceChannel = 0;  
    end  
end
```

10. **Set device configuration to gtecDeviceInterface**

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

11. **Configure g.HIamp with the current settings**

```
gds_interface.SetConfiguration();
```

12. **Start data acquisition and streaming from server**

```
gds_interface.StartDataAcquisition();  
gds_interface.StartStreaming();
```

13. **Get data acquired from g.HIamp, 2560 samples for 10 seconds of data**

```
samples_acquired = 0;  
while (samples_acquired < 2560)  
    [scans_received, data] = gds_interface.GetData(8);
```



```

        samples_acquired = samples_acquired + scans_received;
    end
    scans_received holds the number of actual scans returned by GetData

```

14. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```

gds_interface.StopStreaming();
gds_interface.StopDataAcquisition();
delete(gds_interface);
clear gds_interface;

```

This example is also available as MATLAB script gHIampAPIDemo1.m in the folder
C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gHIamp

4.3 g.USBAMP

To test g.NEEDaccess MATLAB API with g.USBamp, please perform the following example:

1. Create a gtecDeviceInterface object with

```
gds_interface = gtecDeviceInterface;
```

2. Set the IP and ports for the host and the client. If used on a local computer, set the following parameters

```

gds_interface.IPAddressHost = '127.0.0.1';
gds_interface.IPAddressLocal = '127.0.0.1';
gds_interface.HostPort = 50223;
gds_interface.LocalPort = 50224;

```

3. Get the device(s) currently connected

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. Use gUSBampDeviceConfiguration to create a g.USBamp device configuration and, as there should only be one g.USBamp device available, set the device serial as name

```

gusbamp_config = gUSBampDeviceConfiguration();
gusbamp_config.Name = connected_devices(1,1).Name;

```

5. To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

6. Get available channels

```
available_channels = gds_interface.GetAvailableChannels();
```

7. Set the sampling rate to 256 Hz and number of scans to 8

```

gusbamp_config.SamplingRate = 256;
gusbamp_config.NumberOfScans = 8;

```

8. Enable and configure internal signal generator to acquire g.USBamps internal test signal

```

gusbamp_siggen = gUSBampInternalSignalGenerator();
gusbamp_siggen.Enabled = true;
gusbamp_siggen.Frequency = 10;
gusbamp_siggen.WaveShape = 3;
gusbamp_siggen.Amplitude = 200;
gusbamp_siggen.Offset = 0;
gusbamp_config.InternalSignalGenerator = gusbamp_siggen;

```

9. Record all channels available

```

for i=1:size(gusbamp_config.Channels,2)
    if (available_channels(1,i))
        gusbamp_config.Channels(1,i).Available = true;
        gusbamp_config.Channels(1,i).Acquire = true;
        % do not use filters
        gusbamp_config.Channels(1,i).BandpassFilterIndex = -1;
        gusbamp_config.Channels(1,i).NotchFilterIndex = -1;
        % do not use a bipolar channel
        gusbamp_config.Channels(1,i).BipolarChannel = 0;
    end
end

```

10. Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfiguration = gusbamp_config;
```

11. Configure g.Hlamp with the current settings

```
gds_interface.SetConfiguration();
```

12. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();
gds_interface.StartStreaming();
```

13. Get data acquired from g.USBamp, 2560 samples for 10 seconds of data

```

samples_acquired = 0;
while (samples_acquired < 2560)
    [scans_received, data] = gds_interface.GetData(8);
    samples_acquired = samples_acquired + scans_received;
end

```

scans_received holds the number of actual scans returned by GetData

14. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```

gds_interface.StopStreaming();
gds_interface.StopDataAcquisition();
delete(gds_interface);
clear gds_interface;

```

This example is also available as MATLAB script gUSBampAPIDemo1.m in the folder

C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gUSBamp

5 DIGITAL INPUTS

5.1 g.NAUTILUS



NOTE

The example contained in the g.Nautilus examples installation folder (default C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNautilus) is using a 32 channel g.Nautilus device. If another device is used (8, 16 or 64 channels), the code has to be adapted correspondingly (see comments in the example code).

When enabled, digital inputs of g.Nautilus are recorded as additional channel ranging from 0 to 255. To record analog channels and digital inputs of g.Nautilus please perform the following steps:

1. **Create a gtecDeviceInterface object with**

```
gds_interface = gtecDeviceInterface;
```

2. **Set the IP and ports for the host and the client. If used on a local computer, set the following parameters**

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. **Get the device(s) currently connected**

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. **Use gNautilusDeviceConfiguration to create a g.Nautilus device configuration and, as there should only be one g.Nautilus device available, set the device serial as name**

```
gnautilus_config = gNautilusDeviceConfiguration();  
gnautilus_config.Name = connected_devices(1,1).Name;
```

5. **To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object**

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

6. **Get available channels, supported sensitivities and supported input sources**

```
available_channels = gds_interface.GetAvailableChannels();  
supported_sensitivities = gds_interface.GetSupportedSensitivities();  
supported_input_sources = gds_interface.GetSupportedInputSources();
```

7. **Set the sampling rate to 250 Hz and number of scans to 8**

```
gnautilus_config.SamplingRate = 250;  
gnautilus_config.NumberOfScans = 8;
```

8. **Set input source to record the internal test signal of g.Nautilus, a square wave applied to all analog channels with a frequency of 10 Hz and an amplitude of 15 mV**

```
gnautilus_config.InputSource = supported_input_source(3).Value;
```

9. **Enable digital inputs for recording**

```
gnautilus_config.DigitalIOs = true;
```

10. Enable additional channels counter and validation indicator

```
gnautilus_config.Counter = true;  
gnautilus_config.ValidationIndicator = true;
```

11. Record all channels available

```
for i=1:size(gNautilusConfig.Channels,2)  
    if (available_channels(1,i))  
        gnautilus_config.Channels(1,i).Available = true;  
        gnautilus_config.Channels(1,i).Acquire = true;  
        % set sensitivity to 187.5 mV  
        gnautilus_config.Channels(1,i).Sensitivity = ...  
            supported_sensitivities(6);  
        % do not use channel for CAR and noise reduction  
        gnautilus_config.Channels(1,i).UsedForNoiseReduction = false;  
        gnautilus_config.Channels(1,i).UsedForCAR = false;  
        % do not use filters  
        gnautilus_config.Channels(1,i).BandpassFilterIndex = -1;  
        gnautilus_config.Channels(1,i).NotchFilterIndex = -1;  
        % do not use a bipolar channel  
        gnautilus_config.Channels(1,i).BipolarChannel = -1;  
    end  
end
```

12. Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

13. Configure g.Nautilus with the current settings

```
gds_interface.SetConfiguration();
```

14. Start data acquisition and streaming from serve

```
gds_interface.StartDataAcquisition();  
gds_interface.StartStreaming();
```

15. Get data acquired from g.Nautilus, 2500 samples for 10 seconds of data

```
samples_acquired = 0;  
while (samples_acquired < 2500)  
    [scans_received, data] = gds_interface.GetData(8);  
    samples_acquired = samples_acquired + scans_received;  
end
```

16. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```
gds_interface.StopStreaming();  
gds_interface.StopDataAcquisition();  
delete(gds_interface);  
clear gds_interface;
```

This example is also available as MATLAB script gNautilusAPIDIDemo.m in the folder

C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNautilus

5.2 g.HIAMP

When enabled, digital inputs of g.HIamp are recorded as additional channel ranging from 0 to 65535. To record analog channels and digital inputs of g.HIamp please perform the following steps:

1. Create a gtecDeviceInterface object with:

```
gds_interface = gtecDeviceInterface;
```

2. Set the IP and ports for the host and the client. If used on a local computer, set the following parameters

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. Get the device(s) currently connected

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. Use gHIampDeviceConfiguration to create a g.HIamp device configuration and, as there should only be one g.HIamp device available, set the device serial as name

```
ghiamp_config = gHIampDeviceConfiguration();  
ghiamp_config.Name = connected_devices(1,1).Name;
```

5. To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

6. Get available channels, supported sensitivities and supported input sources

```
available_channels = gds_interface.GetAvailableChannels();
```

7. Set the sampling rate to 256 Hz and number of scans to 8

```
ghiamp_config.SamplingRate = 256;  
ghiamp_config.NumberOfScans = 8;
```

8. Enable digital inputs for recording

```
ghiamp_config.TriggerLinesEnabled = true;
```

9. Enable counter

```
ghiamp_config.CounterEnabled = true;
```

10. Record all channels available

```
for i=1:size(ghiamp_config.Channels,2)  
    if (available_channels(1,i))  
        ghiamp_config.Channels(1,i).Available = true;  
        ghiamp_config.Channels(1,i).Acquire = true;  
        % do not use filters  
        ghiamp_config.Channels(1,i).BandpassFilterIndex = -1;  
        ghiamp_config.Channels(1,i).NotchFilterIndex = -1;  
        % do not use a bipolar channel  
        ghiamp_config.Channels(1,i).ReferenceChannel = 0;  
    end  
end
```

11. Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

12. Configure g.Hlamp with the current settings

```
gds_interface.SetConfiguration();
```

13. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();  
gds_interface.StartStreaming();
```

14. Get data acquired from g.Hlamp, 2560 samples for 10 seconds of data

```
samples_acquired = 0;  
while (samples_acquired < 2560)  
    [scans_received, data] = gds_interface.GetData(8);  
    samples_acquired = samples_acquired + scans_received;  
end
```

15. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```
gds_interface.StopStreaming();  
gds_interface.StopDataAcquisition();  
delete(gds_interface);  
clear gds_interface;
```

This example is also available as MATLAB script gHlampAPIDemo.m in the folder
C:\Users\p

5.3 g.USBAMP

When enabled, digital inputs of g.USBamp are recorded as additional channel ranging from 0 to 255. To record analog channels and digital inputs of g.USBamp please perform the following steps:

1. Create a gtecDeviceInterface object with:

```
gds_interface = gtecDeviceInterface;
```

2. Set the IP and ports for the host and the client. If used on a local computer, set the following parameters

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. Get the device(s) currently connected

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. Use gUSBampDeviceConfiguration to create a g.USBamp device configuration and, as there should only be one g.USBamp device available, set the device serial as name

```
gusbamp_config = gUSBampDeviceConfiguration();  
gusbamp_config.Name = connected_devices(1,1).Name;
```

5. To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object

```
gds_interface.DeviceConfigurations = gusbamp_config;
```

6. Get available channels

```
available_channels = gds_interface.GetAvailableChannels();
```

7. Set the sampling rate to 256 Hz and number of scans to 8

```
gusbamp_config.SamplingRate = 256;
gusbamp_config.NumberOfScans = 8;
```

8. Enable digital inputs for recording

```
gusbamp_config.TriggerEnabled = true;
```

9. Record all channels available

```
for i=1:size(gusbamp_config.Channels,2)
    if (available_channels(1,i))
        gusbamp_config.Channels(1,i).Available = true;
        gusbamp_config.Channels(1,i).Acquire = true;
        % do not use filters
        gusbamp_config.Channels(1,i).BandpassFilterIndex = -1;
        gusbamp_config.Channels(1,i).NotchFilterIndex = -1;
        % do not use a bipolar channel
        gusbamp_config.Channels(1,i).ReferenceChannel = 0;
    end
end
```

Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = gusbamp_config;
```

10. Configure g.USBamp with the current settings

```
gds_interface.SetConfiguration();
```

11. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();
gds_interface.StartStreaming();
```

12. Get data acquired from g.USBamp, 2560 samples for 10 seconds of data

```
samples_acquired = 0;
while (samples_acquired < 2560)
    [scans_received, data] = gds_interface.GetData(8);
    samples_acquired = samples_acquired + scans_received;
end
```

13. End streaming and stop data acquisition and delete and clear gtecDeviceInterface

```
gds_interface.StopStreaming();
gds_interface.StopDataAcquisition();
delete(gds_interface);
clear gds_interface;
```

This example is also available as MATLAB script gUSBampAPIDemo.m in the folder

C:\Users\

6 DISPLAY DATA

MATLAB's DSP System Toolbox provides a time scope, which can be used to display acquired data right during data acquisition.

6.1 g.NAUTILUS



NOTE

The example contained in the g.Nautilus examples installation folder (default C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNauilus) is using a 32 channel g.Nautilus device. If another device is used (8, 16 or 64 channels), the code must be adapted correspondingly (see comments in the example code).

To use g.tec MATLAB API with g.Nautilus and the time scope please perform the following steps:

1. **Create a time scope for 3 input channels, 250Hz sampling frequency, a buffer length of 10 seconds (2500 samples per channel) and an amplitude range from 0 to 16 mV.**

2.

```
scope_handle = timescope('SampleRate',250,'BufferLength',2500,'YLimits', [0 16000],...
    'AxesScaling','manual','TimeAxisLabels','Bottom',...
    'LayoutDimensions',[3 1],'YLabel','Amplitude [µV]',...
    'ChannelNames',{'EEG','Counter','Validation Indicator'},...
    'TimeSpan',10);
set(scope_handle, 'ActiveDisplay', 2, 'YLimits', [0 2510], 'YLabel',
'Counter');
set(scope_handle, 'ActiveDisplay', 3, 'YLimits', [0 1], 'YLabel', 'Valid');
```

3. **Create a gtecDeviceInterface object with**

```
gds_interface = gtecDeviceInterface;
```

4. **Set the IP and ports for the host and the client. If used on a local computer, set the following parameters**

```
gds_interface.IPAddressHost = '127.0.0.1';
gds_interface.IPAddressLocal = '127.0.0.1';
gds_interface.HostPort = 50223;
gds_interface.LocalPort = 50224;
```

5. **Get the device(s) currently connected**

```
connected_devices = gds_interface.GetConnectedDevices();
```

6. **Use gNautilusDeviceConfiguration to create a g.Nautilus device configuration and, as there should only be one g.Nautilus device available, set the device serial as name**

```
gnautilus_config = gNautilusDeviceConfiguration();
gnautilus_config.Name = connected_devices(1,1).Name;
```

7. **To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object**

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

8. **Get available channels, supported sensitivities and supported input sources**


```
available_channels = gds_interface.GetAvailableChannels();
supported_sensitivities = gds_interface.GetSupportedSensitivities();
supported_input_sources = gds_interface.GetSupportedInputSources();
```

9. Set the sampling rate to 250 Hz and number of scans to 8

```
gnautilus_config.SamplingRate = 250;
gnautilus_config.NumberOfScans = 8;
```

10. Set input source to record the internal test signal of g.Nautilus, a square wave applied to all analog channels with a frequency of 10 Hz and an amplitude of 15 mV

```
gnautilus_config.InputSource = supported_input_source(3).Value;
```

11. Enable additional channels counter and validation indicator

```
gnautilus_config.Counter = true;
gnautilus_config.ValidationIndicator = true;
```

12. Record all channels available

```
for i=1:size(gNautilusConfig.Channels,2)
    if (available_channels(1,i))
        gnautilus_config.Channels(1,i).Available = true;
        gnautilus_config.Channels(1,i).Acquire = true;
        % set sensitivity to 187.5 mV
        gnautilus_config.Channels(1,i).Sensitivity = ...
            supported_sensitivities(6);
        % do not use channel for CAR and noise reduction
        gnautilus_config.Channels(1,i).UsedForNoiseReduction = false;
        gnautilus_config.Channels(1,i).UsedForCAR = false;
        % do not use filters
        gnautilus_config.Channels(1,i).BandpassFilterIndex = -1;
        gnautilus_config.Channels(1,i).NotchFilterIndex = -1;
        % do not use a bipolar channel
        gnautilus_config.Channels(1,i).BipolarChannel = -1;
    end
end
```

Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = gnautilus_config;
```

13. Configure g.Nautilus with the current settings

```
gds_interface.SetConfiguration();
```

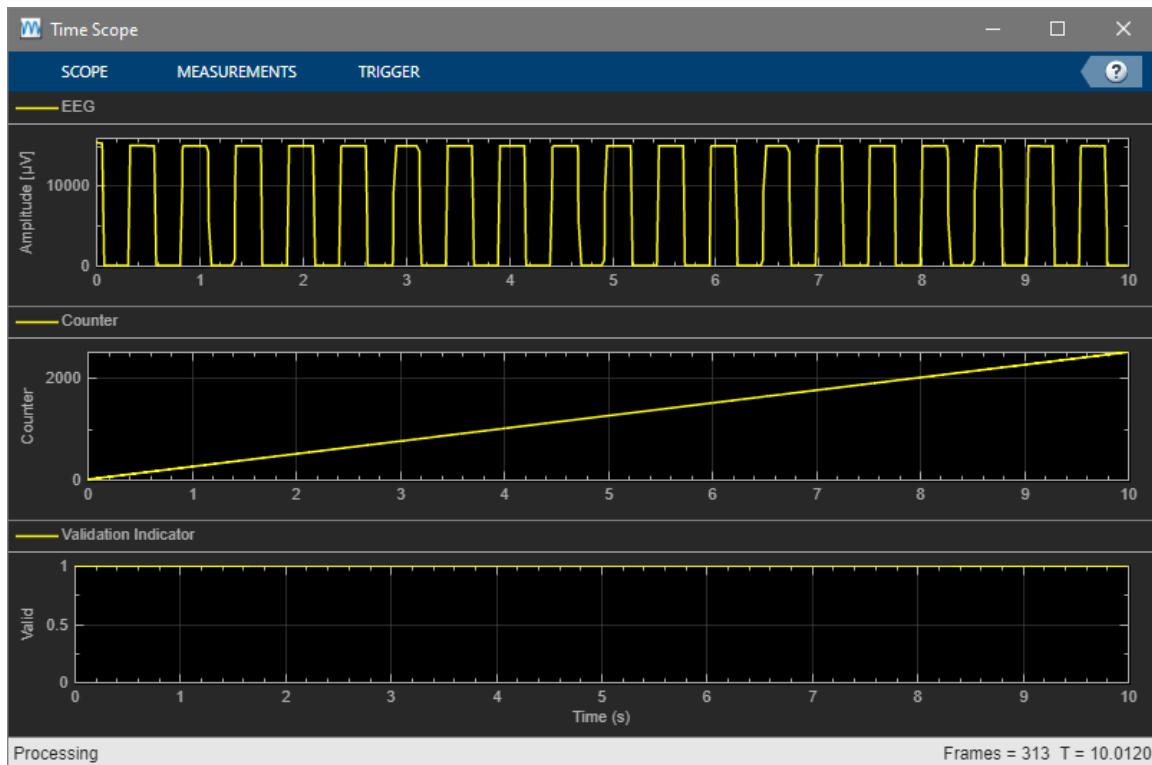
14. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();
gds_iinterface.StartStreaming();
```

15. Get data acquired from g.Nautilus, 2500 samples for 10 seconds of data and display data in the time scope

```
samples_acquired = 0;
while (samples_acquired < 2500)
    [scans_received, data] = gds_interface.GetData(8);
    step(scope_handle, data(:,1),data(:,9),data(:,10));
    samples_acquired = samples_acquired + scans_received;
end
```

16. DSP time scope displays the first analog channel, counter and the validation indicator



17. End streaming and stop data acquisition and delete and clear `gtecDeviceInterface` as well as the time scope handle

```
gds_interface.StopStreaming();
gds_interface.StopDataAcquisition();
delete(gds_interface);
scope_handle.hide;
clear gds_interface;
clear scope_handle;
```

This example is also available as MATLAB script `gNautilusAPIScope.m` in the folder
`C:\Users\\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gNautilus`

6.2 G.HIAMP

To use `gtec` MATLAB API with `g.HIamp` and the time scope please perform the following steps:

18. Create a time scope for 2 input channels, 256Hz sampling frequency, a buffer length of 10 seconds (2560 samples per channel) and an amplitude range from -15 to 0 mV.
19.

```
scope_handle = timescope('SampleRate', 256, 'BufferLength', 2560, 'YLimits',
    [0 2560],...
    'AxesScaling', 'manual', 'TimeAxisLabels','Bottom', 'TimeSpan',10...
    'LayoutDimensions',[2 1], 'ChannelNames', {'Counter','EEG'},...
    'YLabel','Counter');
```

```
set(scope_handle, 'ActiveDisplay', 2, 'YLimits', [-15300 0], 'YLabel', '
    Amplitude [µV]');
```

20. Create a `gtecDeviceInterface` object with

```
gds_interface = gtecDeviceInterface;
```

21. Set the IP and ports for the host and the client. If used on a local computer, set the following parameters

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

22. Get the device(s) currently connected

```
connected_devices = gds_interface.GetConnectedDevices();
```

23. Use gHIampDeviceConfiguration to create a g.HIamp device configuration and, as there should only be one g.HIamp device available, set the device serial as name

```
ghiamp_config = gHIampDeviceConfiguration();  
ghiamp_config.Name = connected_devices(1,1).Name;
```

24. To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

25. Get available channels, supported sensitivities and supported input sources

```
available_channels = gds_interface.GetAvailableChannels();
```

26. Set the sampling rate to 256 Hz and number of scans to 8

```
ghiamp_config.SamplingRate = 256;  
ghiamp_config.NumberOfScans = 8;
```

27. Use the internal test signal of gHIamp, a square wave applied to all analog channels with a frequency of 10 Hz and an amplitude of 7.5 mV and an offset of -7.5mV, by enabling the corresponding setting

```
ghiamp_siggen = gHIampInternalSignalGenerator();  
ghiamp_siggen.Enabled = true;  
ghiamp_siggen.Frequency = 10;  
ghiamp_config.InternalSignalGenerator = ghiamp_siggen;
```

28. Enable counter

```
ghiamp_config.CounterEnabled = true;
```

29. Record all channels available

```
for i=1:size(ghiamp_config.Channels,2)  
    if (available_channels(1,i))  
        ghiamp_config.Channels(1,i).Available = true;  
        if (i <= 80)  
            ghiamp_config.Channels(1,i).Acquire = true;  
        else  
            ghiamp_config.Channels(1,i).Acquire = false;  
        end  
        % do not use filters  
        ghiamp_config.Channels(1,i).BandpassFilterIndex = -1;  
        ghiamp_config.Channels(1,i).NotchFilterIndex = -1;  
        % do not use a bipolar channel  
        ghiamp_config.Channels(1,i).ReferenceChannel = 0;  
    end  
end
```

30. Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfigurations = ghiamp_config;
```

31. Configure g.Hlamp with the current settings

```
gds_interface.SetConfiguration();
```

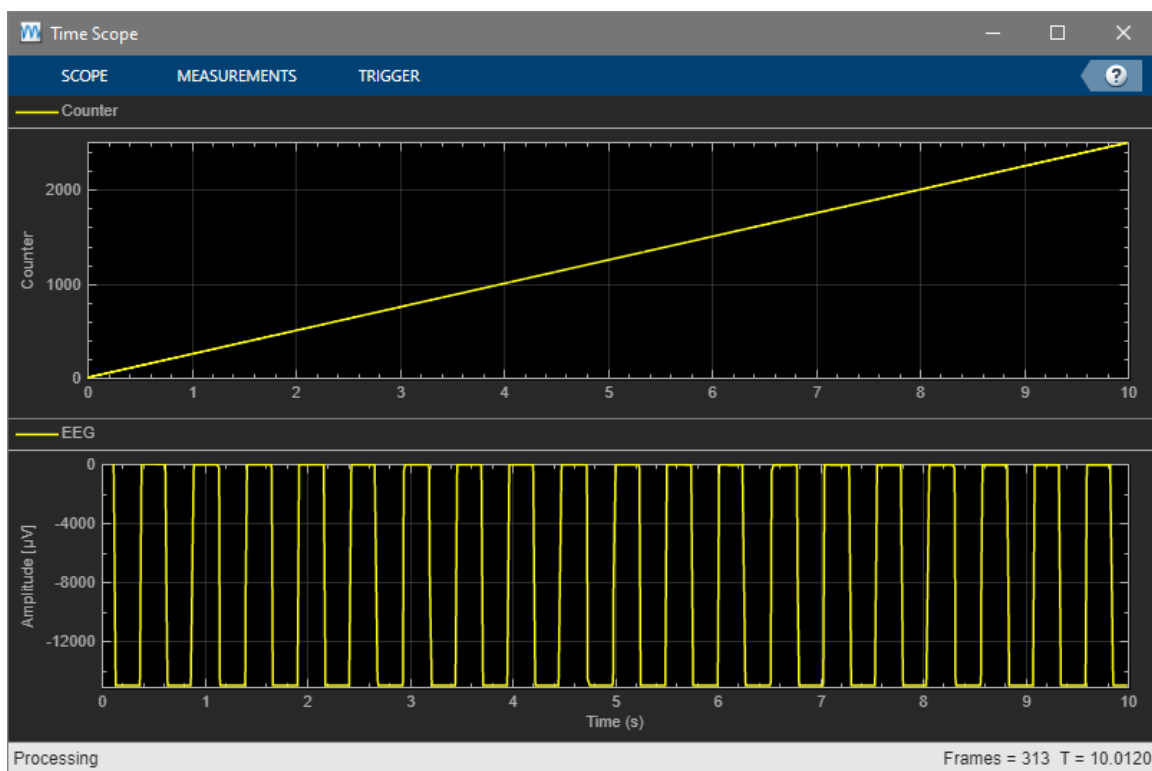
32. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();  
gds_interface.StartStreaming();
```

33. Get data acquired from g.Hlamp, 2560 samples for 10 seconds of data and display data in the time scope

```
samples_acquired = 0;  
while (samples_acquired < 2560)  
    [scans_received, data] = gds_interface.GetData(8);  
    step(scope_handle, data(:,1), data(:,2));  
    samples_acquired = samples_acquired + scans_received;  
end
```

34. DSP time scope displays the counter and the first analog channel



35. End streaming and stop data acquisition and delete and clear gtecDeviceInterface as well as the time scope handle

```
gds_interface.StopStreaming();  
gds_interface.StopDataAcquisition();  
delete(gds_interface);  
scope_handle.hide;  
clear gds_interface;  
clear scope_handle;
```

This example is also available as MATLAB script gHlampAPIScope.m in the folder

C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gHlamp

6.3 G.USBAMP

To use g.tec MATLAB API with g.USBamp and the time scope please perform the following steps:

1. **Create a time scope for 2 input channels, 256Hz sampling frequency, a buffer length of 10 seconds (2560 samples per channel) and an amplitude range from -200 to 200 mV.**

```
scope_handle = dsp.TimeScope(2, 256, 'BufferLength', 2560,...  
    'TimeAxisLabels', 'Bottom', 'YLimits', [-200000 100000],...  
    'TimeSpan', 10, 'LayoutDimensions', [2,1], 'ReduceUpdates', true,...  
    'YLabel', 'Amplitude [µV]');
```

1. **Create a gtecDeviceInterface object with**

```
gds_interface = gtecDeviceInterface;
```

2. **Set the IP and ports for the host and the client. If used on a local computer, set the following parameters**

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

3. **Get the device(s) currently connected**

```
connected_devices = gds_interface.GetConnectedDevices();
```

4. **Create a gUSBampDeviceConfiguration and, as there should only be one g.USBamp device available, set the device serial as name**

```
gusbamp_config = gUSBampDeviceConfiguration();  
gusbamp_config.Name = connected_devices(1,1).Name;
```

5. **To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object**

```
gds_interface.DeviceConfigurations = gusbamp_config;
```

6. **Get available channels**

```
available_channels = gds_interface.GetAvailableChannels();
```

7. **Set the sampling rate to 256 Hz and number of scans to 8**

```
gusbamp_config.SamplingRate = 256;  
gusbamp_config.NumberOfScans = 8;
```

8. **Activate the internal test signal of g.USBamp, a square wave applied to all analog channels with a frequency of 10 Hz and amplitude of 200 mV. Shortcut analog channel 1 to ground.**

```
gusbamp_siggen = gUSBampInternalSignalGenerator();  
gusbamp_siggen.Enabled = true;  
gusbamp_siggen.Frequency = 10;  
gusbamp_siggen.WaveShape = 3;  
gusbamp_siggen.Amplitude = 200;  
gusbamp_siggen.Offset = 0;  
gusbamp_config.InternalSignalGenerator = gusbamp_siggen;
```

9. **Disable counter**

```
gusbamp_config.CounterEnabled = false;
```

10. **Record all channels available**

```
for i=1:size(gusbamp_config.Channels,2)  
    if (available_channels(1,i))  
        gusbamp_config.Channels(1,i).Available = true;  
        if (i <= 80)
```

```

        ghiamp_config.Channels(1,i).Acquire = true;
    else
        ghiamp_config.Channels(1,i).Acquire = false;
    end
    % do not use filters
    ghiamp_config.Channels(1,i).BandpassFilterIndex = -1;
    ghiamp_config.Channels(1,i).NotchFilterIndex = -1;
    % do not use a bipolar channel
    ghiamp_config.Channels(1,i).ReferenceChannel = 0;
end
end

```

Set device configuration to gtecDeviceInterface

```
gds_interface.DeviceConfiguration = gusbamp_config;
```

11. Configure g.USBamp with the current settings

```
gds_interface.SetConfiguration();
```

12. Start data acquisition and streaming from server

```
gds_interface.StartDataAcquisition();
gds_interface.StartStreaming();
```

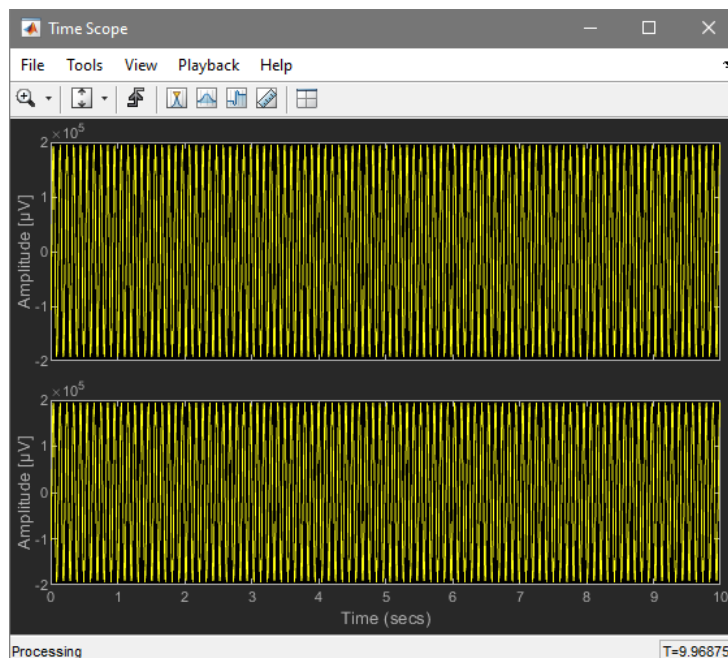
13. Get data acquired from g.USBamp, 2560 samples for 10 seconds of data and display data in the time scope

```

samples_acquired = 0;
while (samples_acquired < 2560)
    [scans_received, data] = gds_interface.GetData(8);
    step(scope_handle, data(:,1), data(:,16));
    samples_acquired = samples_acquired + scans_received;
end

```

14. DSP time scope displays the first two analog channels.



15. End streaming and stop data acquisition and delete and clear gtecDeviceInterface as well as the time scope handle

```
gds_interface.StopStreaming();  
gds_interface.StopDataAcquisition();  
delete(gds_interface);  
scope_handle.hide;  
clear gds_interface;  
clear scope_handle;
```

This example is also available as MATLAB script `gUSBampAPIScope.m` in the folder

`C:\Users\<user_name>\Documents\gttec\gNEEDaccessMATLABAPI\Examples\gUSBamp`

7 SYNCHRONIZING g.USBAMP

This demo shows how to synchronize two g.USBamps with the g.NEEDaccess MATLAB API and display data in the DSP time scope. It requires two amplifiers and a cable to synchronize the two devices.



NOTE

As the GetConnectedDevices function of the gtecDeviceInterface object returns the connected g.USBamps as they are recognized by GDS, it might be useful to check the order in the returned object (steps 2 to 4 in this section) separately in the MATLAB command window before running the demo. Adapt the code described in step 5 afterwards accordingly.

The following steps are required to execute this demo:

1. Create the DSP time scope with two axes for displaying data during acquisition

```
scope_handle = dsp.TimeScope(2,256, 'BufferLength', 2560,...  
'YLimits', [-200000 200000], 'TimeSpan', 5, 'LayoutDimensions', [2,1],...  
'ReduceUpdates',true, 'YLabel','Amplitude [µV]');  
set(scope_handle, 'ActiveDisplay',2, 'YLimits', [-200000 200000],  
'YLabel','Amplitude [µV]');
```

2. Create a gtecDeviceInterface object with:

```
gds_interface = gtecDeviceInterface;
```

3. Set the IP and ports for the host and the client. If used on a local computer, set the following parameters

```
gds_interface.IPAddressHost = '127.0.0.1';  
gds_interface.IPAddressLocal = '127.0.0.1';  
gds_interface.HostPort = 50223;  
gds_interface.LocalPort = 50224;
```

4. Get the device(s) currently connected

```
connected_devices = gds_interface.GetConnectedDevices();
```

5. Use gUSBampDeviceConfiguration to create an array of two g.USBamp device configurations and, as there should only be two g.USBamp devices available, set the device serials as names. Make sure that first device is master, second device must be slave.

```
gusbamp_configs(1,1:2) = gUSBampDeviceConfiguration();  
gusbamp_configs(1,1).Name = connected_devices(1,1).Name;  
gusbamp_configs(1,2).Name = connected_devices(1,2).Name;
```

6. To use functions requiring a device serial later, set this newly created device configuration to the gtecDeviceInterface object

```
gds_interface.DeviceConfigurations = gusbamp_configs;
```

7. Get available channels

```
available_channels_master =  
gds_interface.GetAvailableChannels(connected_devices(1,1).Name);  
available_channels_slave =  
gds_interface.GetAvailableChannels(connected_devices(1,2).Name);
```


- 8. Edit both configurations to have the same sampling rate (256 Hz) and number of scans (8 scans) (will not work if configured differently), enable and configure internal test signal as shown below**

```
for i=1:size(gusbamp_configs,2)
    gusbamp_configs(1,i).SamplingRate = 256;
    gusbamp_configs(1,i).NumberOfScans = 8;
    gusbamp_siggen = gUSBampInternalSignalGenerator();
    gusbamp_siggen.Enabled = true;
    gusbamp_siggen.Frequency = 10;
    gusbamp_siggen.WaveShape = 3;
    gusbamp_siggen.Amplitude = 200;
    gusbamp_siggen.Offset = 0;
    gusbamp_configs(1,i).InternalSignalGenerator = gusbamp_siggen;
    for j=1:size(gusbamp_configs(1,i).Channels,2)
        if (available_channels_master(1,j))
            gusbamp_configs(1,i).Channels(1,j).Available = true;
            gusbamp_configs(1,i).Channels(1,j).Acquire = true;
            % do not use filters
            gusbamp_configs(1,i).Channels(1,j).BandpassFilterIndex = -1;
            gusbamp_configs(1,i).Channels(1,j).NotchFilterIndex = -1;
            % do not use a bipolar channel
            gusbamp_configs(1,i).Channels(1,j).BipolarChannel = 0;
        end
    end
end
```

- 9. Set device configuration to gtecDeviceInterface**

```
gds_interface.DeviceConfigurations = gusbamp_configs;
```

- 10. Configure both amplifiers with the current settings**

```
gds_interface.SetConfiguration();
```

- 11. Start data acquisition and streaming from server**

```
gds_interface.StartDataAcquisition();
gds_interface.StartStreaming();
```

- 12. Get data acquired from both devices, 2560 samples for 10 seconds of data and display data in the time scope**

```
samples_acquired = 0;
while (samples_acquired < 2560)
    [scans_received, data] = gds_interface.GetData(8);
    % display first analog channel for each device in time scope
    step(scope_handle, data(:,1), data(:,17));
end
```

- 13. End streaming and stop data acquisition and delete and clear gtecDeviceInterface as well as the time scope handle**

```
gds_interface.StopStreaming();
gds_interface.StopDataAcquisition();
delete(gds_interface);
scope_handle.hide;

clear gds_interface;
clear scope_handle;
```

This example is also available as MATLAB script gUSBampAPISynchDemo.m in the folder

C:\Users\<user_name>\Documents\gtec\gNEEDaccessMATLABAPI\Examples\gUSBamp

8 HELP

g.NEEDaccess MATLAB API provides a printable documentation.

The printable documentation is stored under

`C:\Program Files\gttec\gNEEDaccessMATLABAPI\Help`

as gNEEDaccessMATLABAPI.pdf. Use Acrobat Reader to view the documentation.

g.NEEDACCESS
NETWORK ENABLED EASY DATA ACCESS